论文解读 · arXiv 2611.08449 · 首次提交 2026-11-05

VFlash

用 Flash Primitive Decoding 实现又快又准的视觉规划

VFlash: Fast, Accurate Visual Planning via Flash Primitive Decoding

Yu-Cheng Chou, Shihao Wang, Jin Huang, Chi-Pin Huang, … , Alan Yuille, Linxi “Jim” Fan, Jan Kautz, Zhiding Yu

一句话总结

VFlash 先用 tool call 声明视觉基元的类型和长度，再让轻量 drafter 一次前向生成整个框 / 轨迹 / 点运动，结构合法就直接接受，不要求与自回归模型逐 token 一致。这样解码提速 12–42×，3D 轨迹比 AR 更准，下游 WAM 成功率从 62.6% 提到 70.2%。

12–42×

解码阶段加速
Thor / Spark / H100， batch = 1

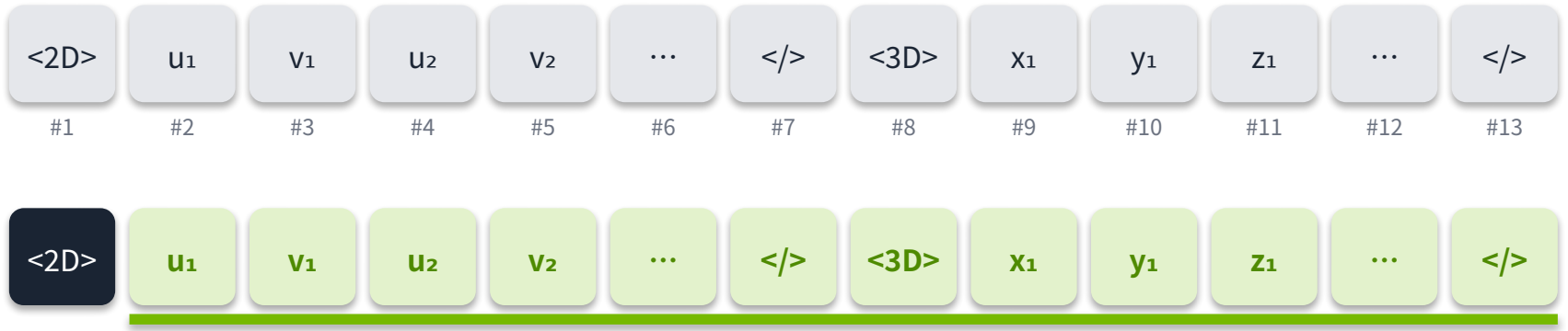
−28% ~ −61%

3D 轨迹 ADE 相对 AR 下降
（2D+3D 目标，三个数据集）

70.2%

MolmoSpaces 平均成功率
基线 WAM 为 62.6%

问题：自回归逐 token 生成坐标，是具身 VLM 视觉规划的延迟瓶颈



示意： start token 已知，其余位置一次性并行填满（实际双臂 10 路点 2D+3D 块为 54 token ）

54 / 84

token：双臂 2D+3D 轨迹块 / Waymo 驾驶块

35–39 ms

Thor 上每个 token 一次 4B VLM 前向

75–87%

Thor 上 AR 解码占每步端到端延迟

2.2

DFlash 在轨迹块内每次仅接受的 token 数

推测解码（DFlash 等）

要求与目标贪心输出逐 token 一致；坐标差 1 个 bin 即被拒

扩散 LLM

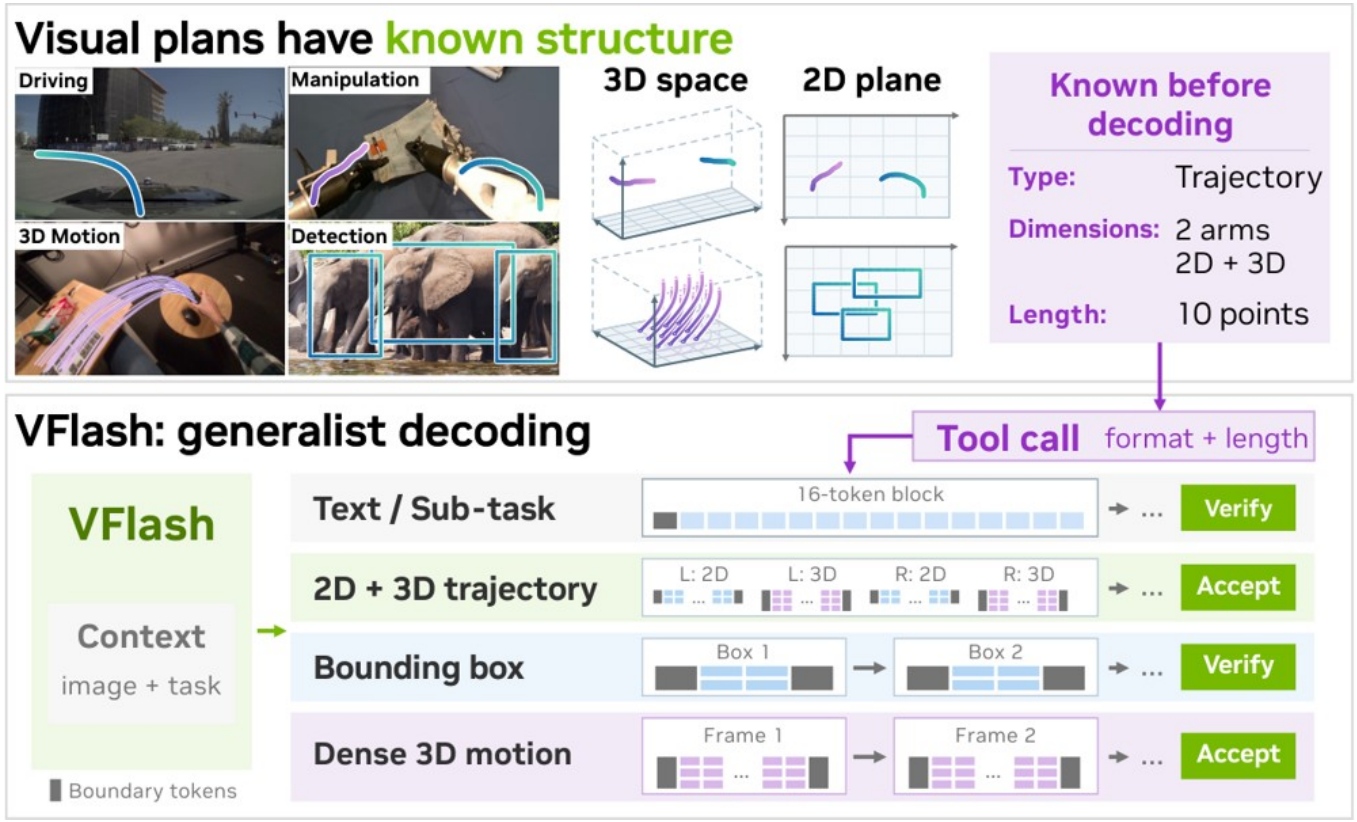
并行去噪，但常有速度–质量取舍

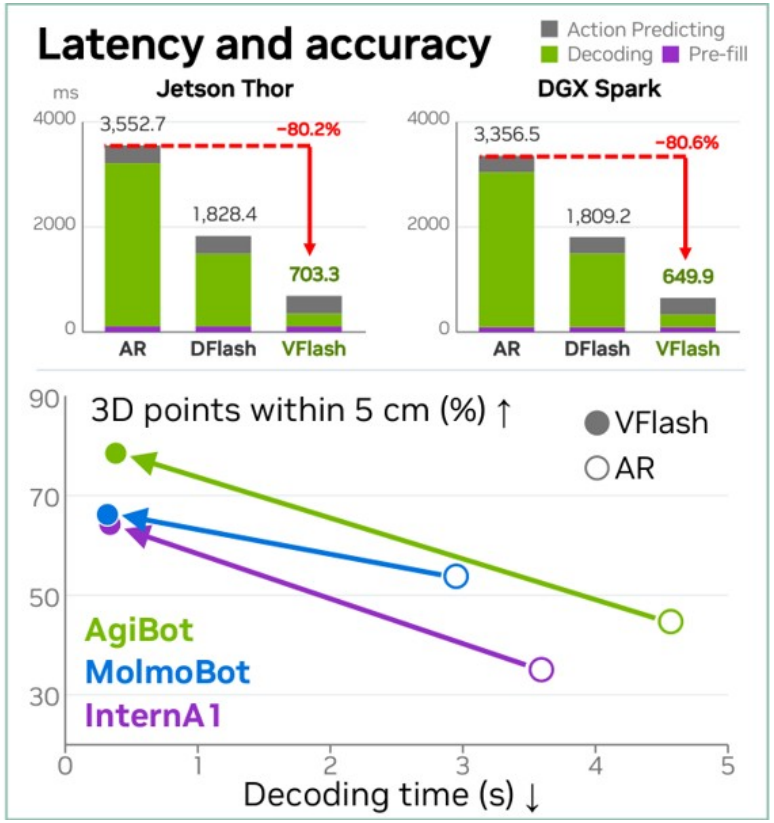
LocateAnything

并行解码框 / 点，但仅限固定大小的 2D 基元

来源：论文 §1、§3.3、附录 A/E

关键洞察：视觉基元的结构在解码前就已知，空间精度也不必复现 AR





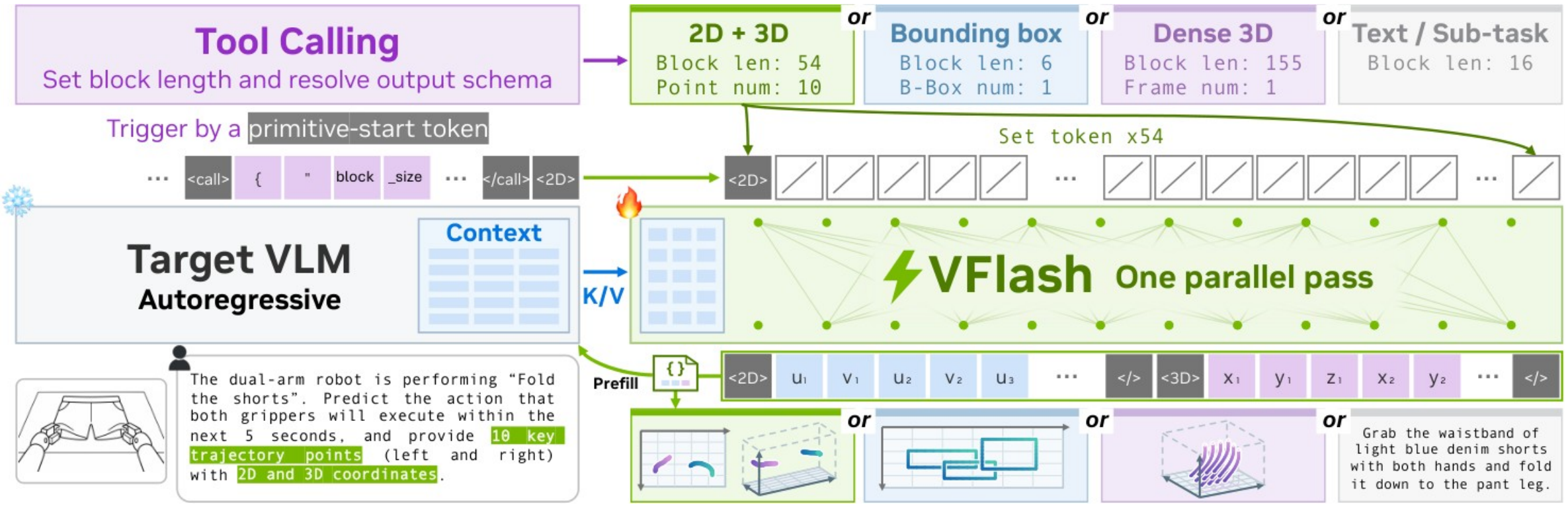
结构已知 → 整块并行

类型 + 维度 + 长度确定后，每个槽位的角色都固定，整个基元可以一次前向预测

精度 ≠ token 一致 → 直接接受

只要结构合法就整块接受；训练也以标注坐标为标签，而不是目标模型的续写

FPD： tool call 声明布局， drafter 一次前向生成完整基元



① Tool call

目标 VLM 以文本写出 block_size，例：轨迹 54 / 框 6 / 稠密 155 / 文本 16

② 布局确定

primitive-start token + 长度 → 每个槽位是路点、坐标轴还是分隔符

③ 一次并行前向

5 层 drafter 以冻结 VLM 第 1/9/17/25/33 层特征为 K/V，填满 B(s)−1 个 mask

④ 直接接受

schema 合法即整块提交；目标 VLM 一次 prefill 更新 KV cache 与动作模型特征

来源：论文 Figure 2、§2.2、附录 A/C

三个设计要点：定宽布局、按标注训练、结构合法即接受

Token 表示

- 边界对齐：一个块 = 一个完整基元
- 位置对齐：每个槽位角色预先确定
- 特殊坐标 token <0>…<1000>， 1 坐标 = 1 位置
- 文本数字目标：符号 +5 位定宽网格，渲染回原格式
- 块长：框 6 ｜单臂 2D+3D 5P+4 ｜双臂 10P+8 ｜驾驶 2D+3D 4P+4 ｜稠密 19P+3

训练

- AR 目标先微调后冻结
- drafter： 5 层 Qwen3 层， 537M 可训练参数，共享目标的嵌入与输出层
- 标签 = 标注坐标，而非目标模型的续写
- 基元内位置等权；文本位置沿用 DFlash 衰减权重
- 同一 drafter 同时服务文本与基元两条路径

推理与接受

- 文本： 16 位 DFlash 块 + 最长前缀校验
- 基元：逐位置限制合法词表， <oof> 配对、框角点顺序等跨位置规则，不增加前向
- 框：额外用 LocateAnything 空间置信度守卫
- 校验失败：回滚到锚点，由受约束的目标 AR 重生成

$\tau_{dir} = B(s)$ 若 $A_s(\hat{a}) = 1$ （结构合法）

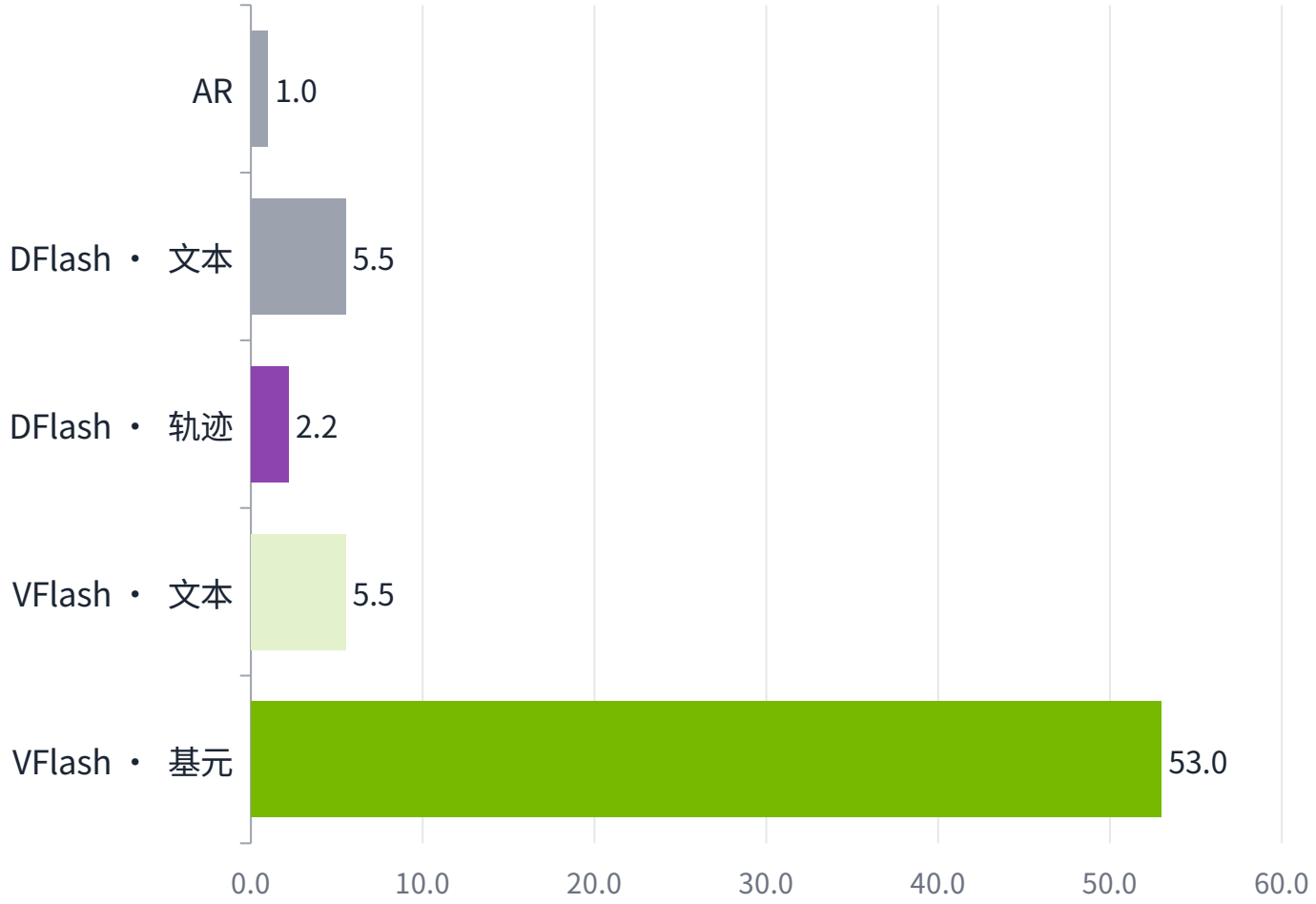
否则 $\tau = 1 + r$，保留前 r 个槽位（轨迹 $r = 0$），其余由受约束 AR 重生成

$L_{dir} = (t_{par} + t_{target}) / B(s)$

每个基元只需 2 次前向：计划越细， B(s) 越大，串行步数不变

逐 token 校验拖慢轨迹： DFlash 在轨迹块内每次仅接受 2.2 个 token

每次迭代提交的 token 数（MolmoSpaces）



Thor 上 MolmoSpaces 解码时间

3107.6 ms

AR： 88.75 次目标前向

1383.3 ms

DFlash： 29.5 次迭代，约减半

258.2 ms

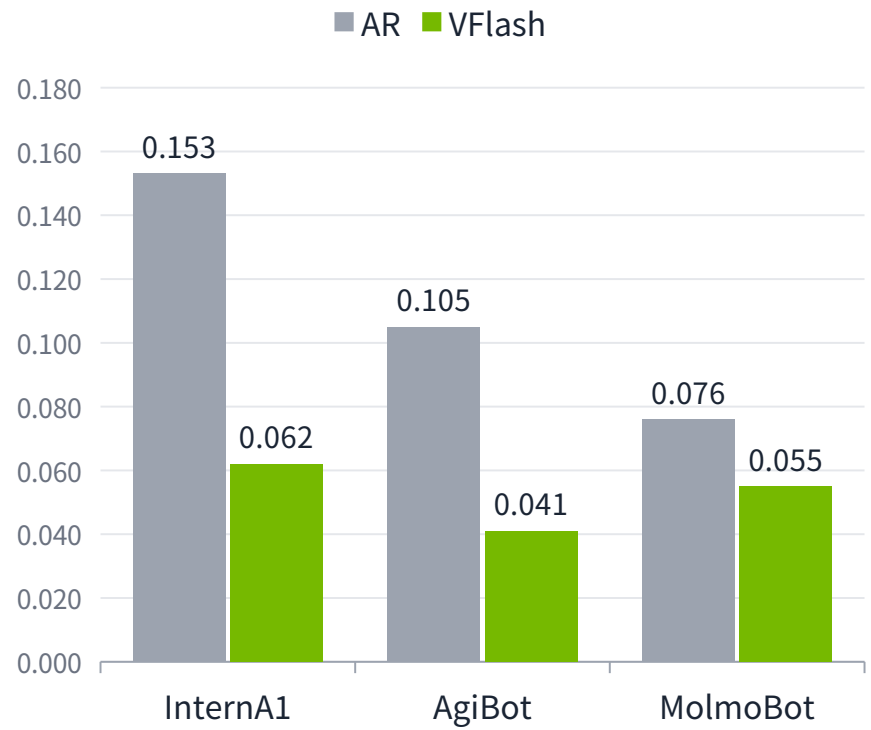
VFlash： 文本 3.25 次 + 基元 1 次

要点：坐标偏一个 bin 就被 Eq.2 拒绝，但几何误差很小。token 一致性 ≠ 空间精度。

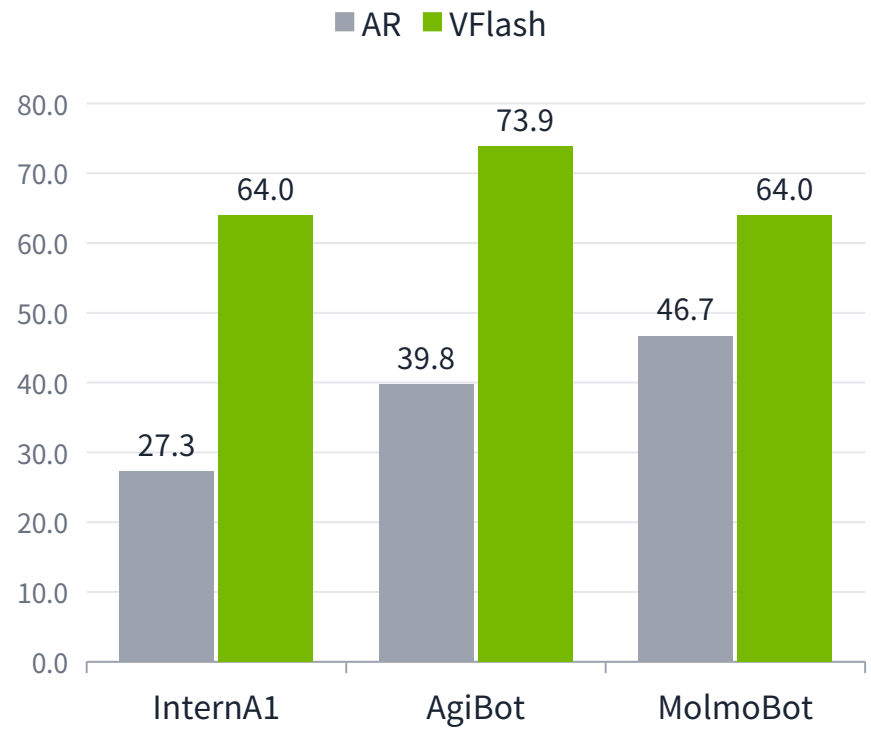
来源：论文 §3.3、附录 E.2 解码统计与分阶段延迟表（MolmoSpaces）

3D 机器人轨迹：一次前向的 drafter 比它的 AR 目标更准

ADE （米，↓）



SR@5cm （%，↑）



−28~61%

2D+3D 目标的 ADE 降幅

−18~49%

3D-only 目标的 ADE 降幅

−3~12%

再加腕部相机训练
（评测仍只用主相机）

观察： AR 在 3D 之外再输出 2D 时，3D ADE 比只输出 3D 高 15–51%；VFlash 一次并行生成 2D+3D，精度基本不变。三个训练变体 × 三个数据集 × 四项指标，VFlash 全部优于 AR。

来源：论文 Table 4 （2D+3D 训练变体）

2D 轨迹全面小幅领先；检测框需 AR 回退才能保住精度

2D 轨迹误差（↓，距离 / 1000）

方法	SR DFD	SR HD	SR RMSE	VA DFD	VA HD	VA RMSE
RoboBrain-2.0 7B	**0.167**	**0.158**	**0.125**	0.329	0.260	0.224
LoHo-Manip 4B	0.231	0.206	0.156	0.212	0.182	0.147
Baseline AR 4B	0.202	0.175	0.142	0.130	0.125	0.093
VFlash 4B	**0.194**	**0.162**	**0.133**	**0.126**	**0.116**	**0.089**

SR = ShareRobot-Bench， VA = VABench-V；绿色为该列最佳

检测框 F1（%，↑）与每秒框数 BPS（H100）

方法	数据集	Mean	@0.50	@0.95	BPS
AR	COCO	**48.68**	65.29	**16.09**	3.15
AR	Dense200	60.10	76.70	**16.95**	6.73
直接接受	COCO	41.32	59.64	10.76	5.96
直接接受	Dense200	39.09	55.06	6.59	**22.66**
AR 回退	COCO	47.79	**66.38**	13.48	8.76
AR 回退	Dense200	**61.78**	**82.64**	12.11	17.46

- 6 项误差全部低于 AR 目标（HD −7%， DFD / RMSE −3~6%）
- VABench-V：比最好的已有方法 LoHo-Manip 低约 40%
- ShareRobot：仅次于 RoboBrain-2.0

- 带 AR 回退： mean F1 与 AR 持平，速度 2.6–2.8×
- F1@0.50 更高（Dense200 82.64 vs 76.70）， F1@0.95 更低
- 无回退： Dense200 mean F1 掉约 20 点，但 BPS 最高

稠密 3D 点运动：单次 rollout 约快 25×，精度有升有降

≈25×

单次 rollout 生成提速
（初步测量， H100 ）

数据集	AR (s)	VFlash (s)	加速
HOT3D	79.8	3.21	**24.8×**
WorldTrack	90.6	3.86	**23.5×**
DAVIS	99.6	3.78	**26.4×**

- 目标：冻结的 MolmoMotion-AR ，仅训练 drafter
- 每帧 8 点 = 一个 19P+3 定宽块；帧间仍串行

PointMotionBench： VFlash vs 目标 MolmoMotion-AR

数据集	方法	ADE ↓	FDE ↓	PWT ↑
HOT3D	MolmoMotion-AR	0.109	0.217	0.444
HOT3D	VFlash	0.122	**0.200**	0.391
WorldTrack	MolmoMotion-AR	0.143	0.261	0.445
WorldTrack	VFlash	0.145	0.300	0.408
DAVIS	MolmoMotion-AR	1.227	2.108	0.153
DAVIS	VFlash	**0.893**	**1.716**	0.143

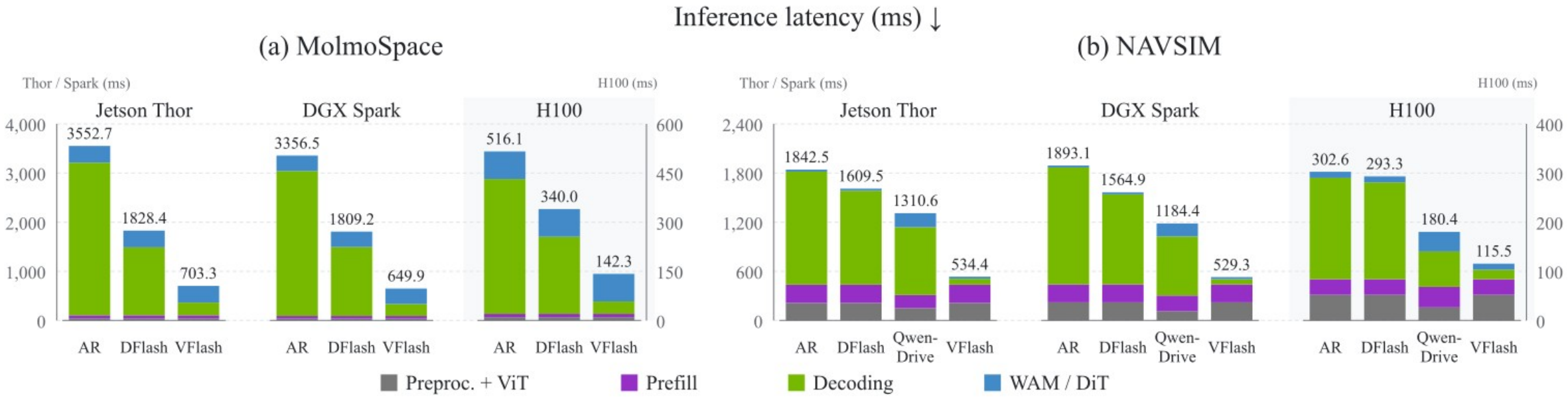
VFlash 行：绿色 = 优于 AR ，红色 = 劣于 AR ； ADE/FDE 单位为米

更好
DAVIS ADE −27% 、 FDE −19% ； HOT3D FDE −8%

更差
WorldTrack FDE +15% ；三个数据集 PWT −7~12%

来源：论文 Table 5 、附录 F.3 （ H100 ， batch = 1 ）

端侧延迟：解码几乎消失， MolmoSpaces 端到端快约 5×

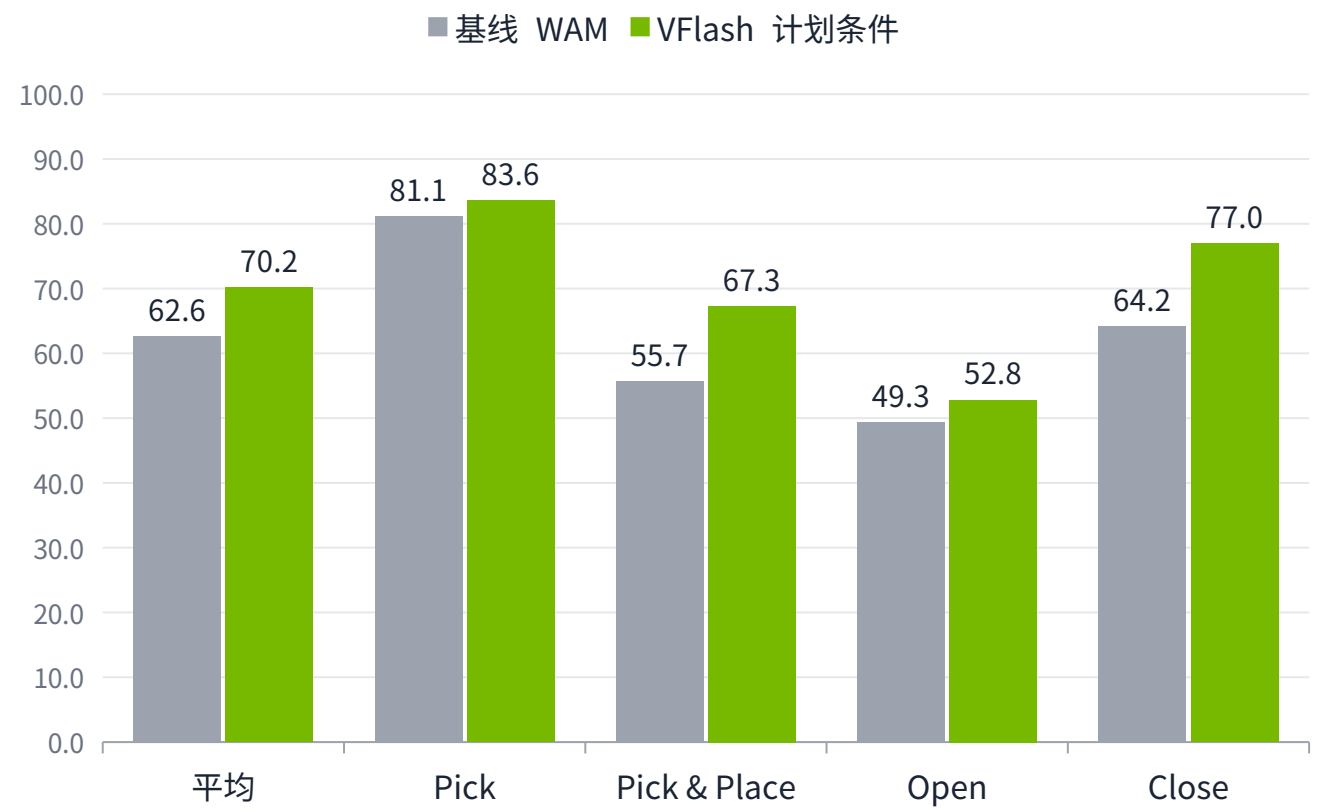


3552.7 → 703.3 MolmoSpaces · Thor （ms）	**≈5.1×**	
3356.5 → 649.9 MolmoSpaces · Spark （ms）	**≈5.2×**	
1842.5 → 534.4 NAVSIM · Thor （ms）	**≈3.4×**	
617.3 → 136.6 Waymo · H100 （ms）	**≈4.5×**	

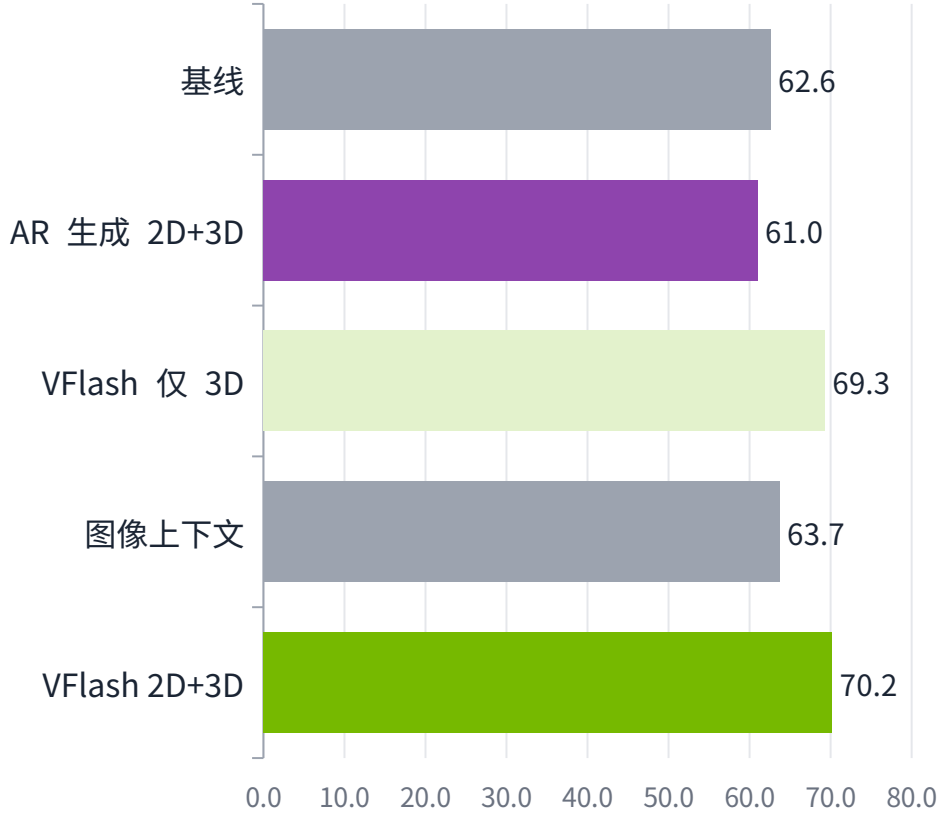
解码阶段缩短 12–42×，其余阶段不变；驾驶基元从 36 增至 84 token， Thor 解码仅从 72 ms 增至 78 ms （AR 下单 token 前向 35–39 ms）。

MolmoSpaces： VFlash 计划让 WAM 平均成功率从 62.6% 提到 70.2%

各任务成功率（%）



消融：计划表示与上下文（平均 %）



+7.6
平均成功率提升（点）

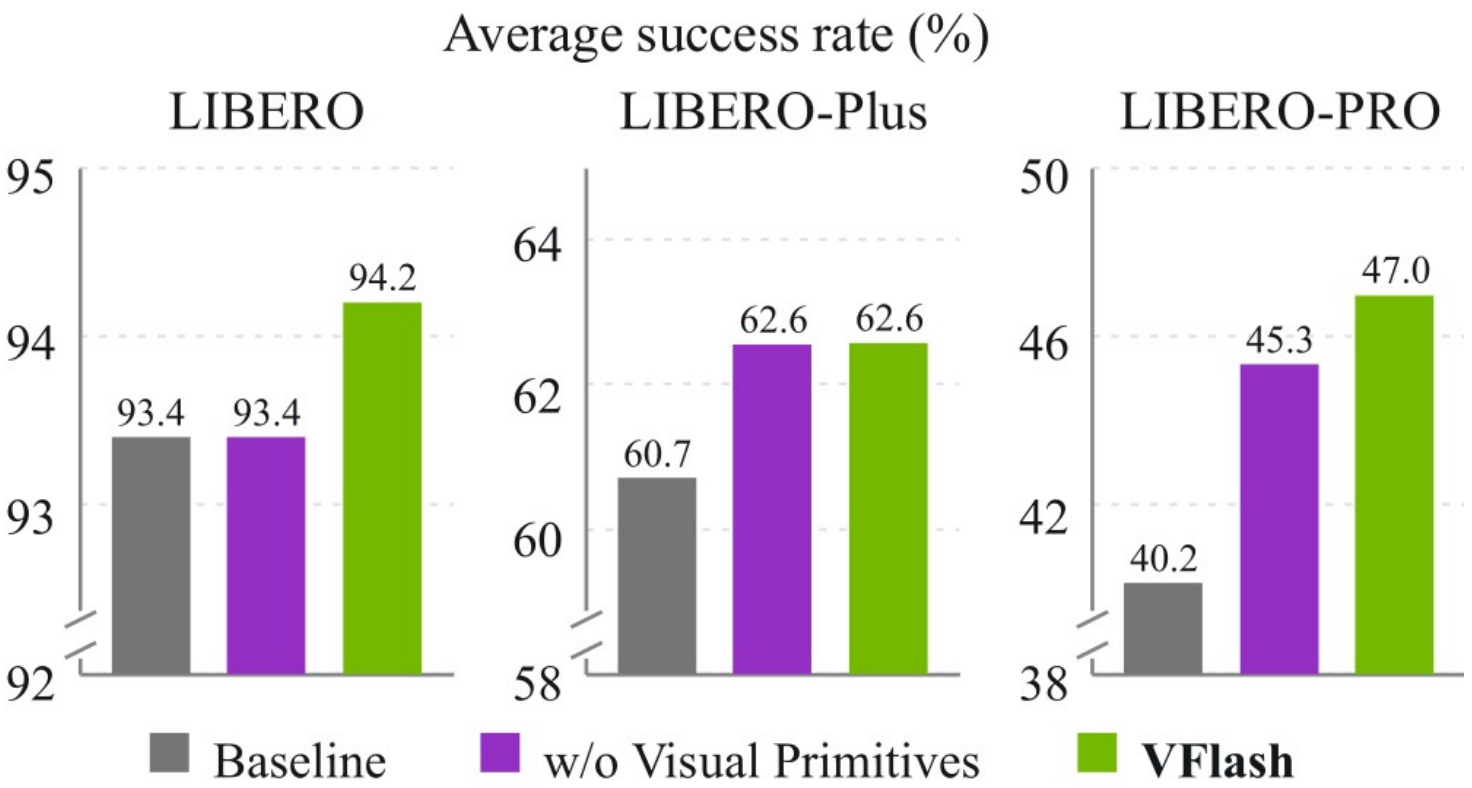
+11.6 / +12.8
Pick&Place / Close 提升

+9.9
高于 PrimeR0 （60.3%）

61.0%
改用 AR 计划：反而低于基线

来源：论文 MolmoSpaces 主结果表与消融表（§3.2）

LIBERO 系列：扰动越强，视觉计划带来的收益越大



相对基线的平均成功率提升

+0.8

LIBERO （93.4 → 94.2）

+1.9

LIBERO-Plus （60.7 → 62.6）

+6.8

LIBERO-PRO （40.2 → 47.0）

- LIBERO-PRO： Spatial / Object / Goal 相对基线 +11.1 / +9.4 / +6.9
- 相对“只用子任务”： Spatial +2.6 、 Goal +1.2 、 Long +2.0 ，轨迹提供子任务之外的信息
- 所有策略只在标准 40 个 LIBERO 任务上训练

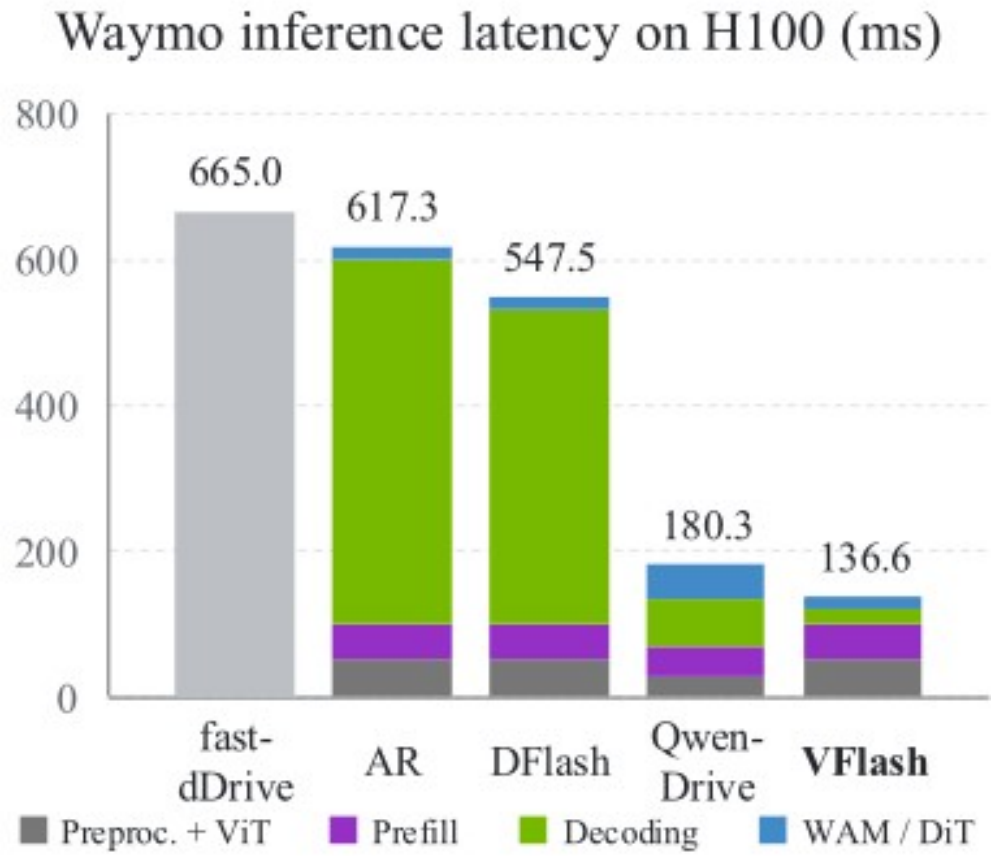
来源：论文 Figure 3 、 §3.2

自动驾驶：冻结 VLM + 小型动作头，成绩有竞争力，延迟更低

NAVSIM PDMS 与 Waymo RFS （↑）

方法	RL	NAVSIM PDMS	Waymo RFS
ReCogDrive	–	86.8	–
ReCogDrive	✓	89.6	–
Qwen-Drive-1.0-SFT	–	88.2	7.78
Qwen-Drive-1.0-RL	✓	**90.7**	**7.91**
VFlash-SFT	–	**87.1**	**7.79**
VFlash-RL	✓	**90.7**	**7.86**

- SFT 已超过 ReCogDrive （87.1 vs 86.8）； RL 只调动作头，计划不变
- NAVSIM 90.7 与 Qwen-Drive-1.0-RL 持平； Waymo 距最佳 7.91 差 0.05
- 注意： Waymo RL checkpoint 在 479 个验证场景上挑选（样本内）



Waymo • H100： 136.6 ms vs AR 617.3 / Qwen-Drive 180.3

Fast-dDrive 665 ms 为其论文报告值，非同环境实测

局限与需要留意的地方

精度并非全面占优

- 框： F1@0.95 下降（COCO 16.09 → 13.48）
- 无回退： Dense200 mean F1 60.10 → 39.09
- 稠密运动： HOT3D / WorldTrack ADE 与三个数据集的 PWT 均落后 AR

对比不完全对称（解读者观点）

- AR 目标只做 next-token 微调； drafter 在同样数据上再训练约 3 个 pass
- 精度优势中，额外训练与并行预测本身各占多少，论文未拆分

下游统计强度有限

- MolmoSpaces / LIBERO： 每个配置仅 1 个训练种子 + 1 个评测种子
- LIBERO 标准集增益仅 +0.8
- Waymo RL checkpoint 在同一批 479 个验证场景上选取（样本内）

测速口径

- 稠密运动 25× 为初步单 rollout 测量，帧间仍串行
- MolmoSpaces AR 解码时间来自早期独立 profile
- tool-call JSON 测速时直接给定，不计入生成

总结：自回归不是视觉输出的质量上限

01

结构即并行度

视觉基元的类型、维度、长度在解码前已知；块与基元对齐后，几十到上百个 token 一次前向生成。

02

不求 token 一致，又快又准

结构合法即接受、按标注坐标训练： 3D 轨迹 ADE 下降 18–61% ，解码快 12–42× 。

03

System-1 式视觉规划

端侧延迟降低约 80% ；以计划为条件的 WAM 在 MolmoSpaces 上从 62.6% 提到 70.2% ， LIBERO-PRO 上 +6.8 。

一句话： VFlash 用“ tool call 声明结构 + 一次前向生成整个基元 + 结构合法即接受”，把具身 VLM 的视觉规划从逐 token 生成变成按基元生成，在轨迹任务上同时做到更快、更准；框和稠密运动的精度取舍，以及单种子的下游实验，还需进一步验证。